

ASPP Coursework 2: Multi-GPU CUDA Wave Equation Solver

Exam No. B281684

Notation. SU/s = site updates per second (product of lattice sites and time-steps per wall-clock second). n_p = number of MPI ranks, also the GPU count in this coursework (one rank per GPU). Throughout, the z axis is the innermost (contiguous) storage dimension.

1 Design and process

Why CUDA, and why two streams. CUDA gives direct control over the two resources that dominate this problem: the launch grid maps 1:1 onto a 3-D index space (no implicit compiler heuristic hides the memory traffic, which matters because the kernel is bandwidth-bound, see §1) and streams + events let us express the exact dependency graph between interior compute, halo pack, MPI, and boundary compute. We use *two* streams because a rank has exactly two executors (one GPU, one CPU): one stream serialises pack and interior, erasing the only overlap opportunity; a third stream would split a chain that has no independent work to place on it.

Exact FLOP count. Counted directly from `step_kernel_interior` and `damped_update` in `wave_cuda.cu`:

Expression	adds	subs	mults	divs	total
Laplacian $\sum_{6 \text{ nbr}} u - 6u_c$	5	1	1	0	7
$v = \text{factor} \cdot c_s^2 \cdot \Delta$	0	0	2	0	2
Undamped tail $2u_c - u_{\text{prev}} + v$	1	1	1	0	3
Interior (undamped) total	6	2	4	0	12
$\text{inv} = 1/(1+d\Delta t)$; $n = 1 - d\Delta t$; $v * = \text{inv}$	1	1	3	1	6
$2\text{inv} u_c - n \text{inv} u_{\text{prev}} + v$	1	1	4	0	6
Boundary (damped) total (7+2+12)	7	3	10	1	21

Damping is only non-zero on the x and y boundary planes, so interior sites dominate and amortised FLOPs/site approach 12 as $L \rightarrow \infty$ (the $\mathcal{O}(L^{-1})$ boundary correction is below 0.1% at $L \geq 1000$). The rest of this report uses **12 FLOPs/site**.

Roofline: both bounds. With 12 FLOPs/site, arithmetic intensities are $I_{\text{opt}} = 0.30$ FLOP/B (optimistic 40 B/site: 4 unique reads + 1 write with L2 reuse) and $I_{\text{naive}} = 0.14$ (88 B/site, 10 reads + 1 write). Device ridges $I_{\text{ridge}} = P_{\text{FP64}}/B_{\text{HBM}}$ are 4.8 (A100) and 10.0 (H100), far above our I . Translated into site throughput:

Ceiling	A100 (SU/s)	H100 (SU/s)
Compute bound $P_{\text{FP64}}/12$	8.08×10^{11}	2.79×10^{12}
Bandwidth bound optimistic $B_{\text{HBM}}/40$	5.10×10^{10}	8.38×10^{10}
Bandwidth bound naive $B_{\text{HBM}}/88$	2.32×10^{10}	3.81×10^{10}
Our best single-GPU (Table 1)	3.56×10^{10}	7.68×10^{10}

We reach 4.4% of the compute bound on A100 and 2.8% on H100, but 70% and 92% of the optimistic bandwidth bound: the problem is unambiguously bandwidth-bound, and optimisation effort targets bytes-per-site.

Interior kernel and block shape. The 32-wide z dimension is exactly one warp, making each neighbour load a coalesced 256 B line: coalescing is all-or-nothing for a memory-bound kernel. A (64,2,4) block, by contrast, spans two warps requesting *the same* x -plane and doubles in-flight L2 requests, costing $\approx 6\%$ on A100 in the `AWAVE_CUDA_BLOCK` sweep; (32,8,2) exceeds the register budget and regresses $\approx 1\%$. The depth-4 x axis empirically gave the best balance between neighbour-plane reuse and occupancy across both GPUs: on H100 the effective traffic is close to half the naive 88 B/site model (Table 1); A100 shows a smaller reduction.

```
// grid = ceil(nz/32) x ceil(ny/4) x ceil(nx/4); block = (32, 4, 4)
k = blockIdx.x*32 + threadIdx.x + 1; // z direction; contiguous / fastest-varying
j = blockIdx.y*4 + threadIdx.y + 1;
i = blockIdx.z*4 + threadIdx.z + 1; // outermost, strided
center = u_now[(i+1)*sx + (j+1)*sy + (k+1)];
lap = sum_6_neighbours - 6*center; // 7 FLOPs
value = factor * cs2[c_idx] * lap; // 2 FLOPs
u_next[...] = (d==0) ? 2*center - u_prev[...] + value // 3 FLOPs
: damped_update(...); // 12 FLOPs
```

Boundary-split kernels. Four kernels (one interior, three face) launch over *disjoint* thread sets. Gains: +4.51% (A100) and +22.18% (H100) at $n_p=2$. The asymmetry is the evidence: H100’s faster SMs issue at a rate where a warp that only evaluates the boundary predicate and early-returns still costs meaningful issue slots; A100, with slower issue, hides that idle time behind other latency. Splitting removes the wasted warps.

Why we did *not* try to fix Z-face coalescing. A natural follow-up question is why we did not special-case the Z-perpendicular boundary face, where the X – Y plane is stored with stride n_z+2 and so is not coalesced. The pack, unpack and `step_kernel_boundary_z_faces` kernels therefore do relatively inefficient DRAM access on that face. We deliberately did not tune it: the boundary work is $\mathcal{O}(L^2)$ while the interior is $\mathcal{O}(L^3)$, and the Z-face stage runs on `halo_stream` concurrently with or immediately after the interior kernel on `compute_stream`. The extra Z-face latency never enters the critical path of a step at the sizes of interest. Introducing a shared-memory transpose or a dedicated coalescing path would add tens of lines of code and complicate the overlap reasoning without reducing the step time. This is an instance of avoiding optimisation off the critical path.

Eight-phase step body. Phases (2) and (3) overlap: pack reads only boundary rows of `d_now`; interior reads only strict interior; the two regions are disjoint so no CUDA sync is needed between streams during a step. Cross-stream events in (8) are device-side at zero CPU cost. The only host-side block is (5); by then the interior kernel is typically already done: on H100 $n_p=4$ at 1000^3 the interior kernel takes about 3.5 ms, while a single `MPI.Waitall` averages around 1.3 ms, and even two such waits per step remain below the interior time.

```
for each step:
  (1) MPI_Irecv on active faces                # host, no GPU dep
  (2) halo_stream: pack_face_{x,y,z}<<<...>>> # reads boundary rows
  (3) compute_stream: step_kernel_interior<<<...>>># disjoint from (2)
  (4) wait pack_done event; MPI_Isend(device ptr)
  (5) MPI_Waitall
  (6) halo_stream: unpack_face_{x,y,z}<<<...>>>
  (7) halo_stream: step_kernel_boundary_{x,y,z}_faces<<<...>>>
  (8) cudaStreamWaitEvent: next-step pack waits for this interior;
      next-step interior waits for this boundary
```

From a CPU/GPU coordination viewpoint, within a step the CPU side only issues light work (posting non-blocking MPI requests and recording/waiting on CUDA events), while the GPU runs the two independent streams in parallel. The CPU blocks exactly once per step, on `MPI.Waitall`, and can do so without stalling the GPU because the compute stream has been fed all of its launches up front. In the common regime where interior time exceeds halo time, this structure means the GPU is continuously busy and the CPU is idle for most of each step.

Rejected optimisations. Every switch with a non-positive mean effect (Appendix A) was removed from the hot code path. *Shared-memory tiling* lost 2.3–4.6%: the working set already lives in L2 on both GPUs, so the extra load, `__syncthreads` and re-read only add latency. *Branchless damping* lost $\approx 0.1\%$ because the interior branch is warp-uniform ($d=0$) and costs nearly nothing; replacing it with an always-executed divide is strictly worse. *Z-padding to 128 B* cost $\approx 0.4\%$: L2 absorbs mis-alignment but padding inflates total DRAM traffic. The *ob1* OpenMPI stack regressed 41% on H100 (loses CUDA IPC). *CUDA Graphs* were also considered and dropped for single-node: their $\approx 10\mu\text{s}$ launch saving is dwarfed by our 1–2 ms interior kernel at 1000^3 ; they become attractive at small per-rank problems on many-node runs (§3). The final code defaults to the best-performing path while retaining a small number of robustness/tuning controls (notably MPI transfer mode and prewarm-iteration count) for portability.

Measurement conventions.

Timing	<code>-nsteps 20, -io 0</code> , no restart, <code>-skip-cpu 1</code> for timing only; 3 repeats, first sample discarded, best steady-state reported.
Correctness	separate runs with CPU reference enabled, <code>ndiff=0</code> required before timing.
MPI layout	most symmetric decomposition at each n_p ($2\times 2\times 2$ at 8, $1\times 2\times 2$ at 4, $1\times 1\times 2$ at 2).
Build	<code>-DCMAKE_BUILD_TYPE=Release -DAWAVE_MODE=CUDA</code> , CUDA 13.0, mainline results use OpenMPI 4.1.9a1 with UCX-enabled CUDA-aware MPI (<code>hcoll/ucc</code> disabled).

2 Results

Environment and largest measurement. Per-GPU hardware: A100-SXM4-80GB / H100-80GB-HBM3; HBM peak 2.04 / 3.35 TB s^{-1} ; L2 40 / 50 MB; FP64 non-sparse 9.7 / 33.5 TFLOP s^{-1} ; NVLink 600 / 900 GB s^{-1} bidirectional [1, 2]. All reported mainline performance points were obtained only after a separate correctness run with the CPU reference enabled (`ndiff=0`). The largest device-path measurement we obtained was **H100** $n_p=8$ at 2500^3 (1.56×10^{10} sites; 4.12×10^{11} SU/s; this point is *reference only*). The unmodified submission cannot accommodate 2500^3 on a single EIDF node because the driver allocates both CPU and GPU mirror states (≈ 96 B

per host point across six arrays times two mirrors, giving ≈ 1.5 TB at $2500^3 \times 8$). For this one point we removed the CPU-mirror allocation from `main.cpp` (valid under `-skip_cpu`) and reconstructed the initial condition on-device in `wave_cuda.cu`; GPU kernels and MPI path are unchanged, so the measurement reflects device-path performance even though the exact workflow is not reproducible from the submitted sources. Details in Appendix C. A100 $n_p=8$ for sizes ≥ 1500 could not be scheduled within the coursework window and is marked N/A.

Single-GPU roofline validation. Inverting measured throughput S to effective traffic $B_{\text{eff}}=B_{\text{HBM}}/S$ gives Table 1. H100 sits at 40.6–43.7 B/site, close to the optimistic bound, consistent with substantial L2 reuse that removes most of the other ≈ 48 B/site in the naive model. A100 lands ≈ 58 B/site: its smaller 40 MB L2 offers less headroom for the same block shape, so neighbour-plane reuse is less complete.

Table 1: Single-GPU roofline validation at $n_p=1$. %-ages are vs the HBM peak at 40 B/site optimistic bound, the 88 B/site naive bound, and $P_{\text{FP64}}/12$ compute bound.

GPU / size	SU/s	B_{eff} (B/site)	% of BW opt	% of BW naive	% of compute
H100 / 256^3	8.26×10^{10}	40.6	98.6%	217%	3.0%
H100 / 512^3	7.67×10^{10}	43.7	91.6%	201%	2.7%
H100 / 1000^3	7.68×10^{10}	43.6	91.7%	202%	2.8%
A100 / 256^3	3.46×10^{10}	59.0	67.8%	149%	4.3%
A100 / 512^3	3.44×10^{10}	59.3	67.4%	148%	4.3%
A100 / 1000^3	3.56×10^{10}	57.3	69.8%	154%	4.4%

H100 exceeding the naive bound is consistent with substantial L2 reuse: the kernel appears to see less DRAM traffic than a 10-reads-1-write model predicts.

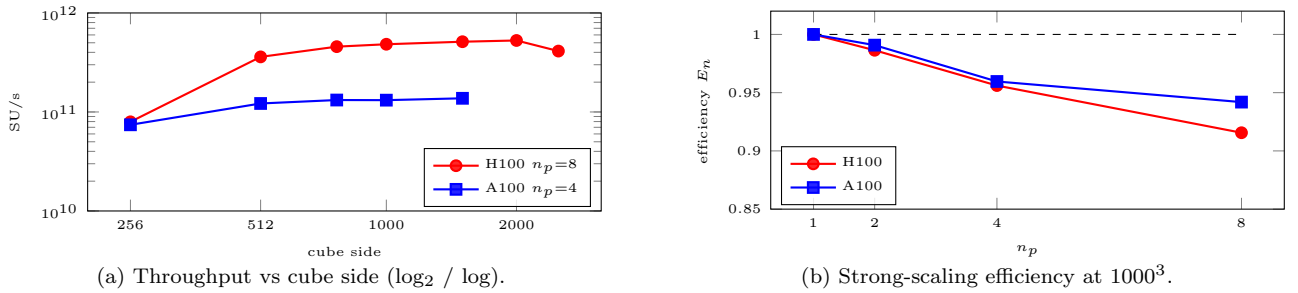


Figure 1: (a) Throughput across problem sizes; A100 $n_p=8$ is omitted from the main curve because its large-size coverage is incomplete and its small-size points fall in a kernel-too-small regime (see text). (b) Strong scaling at 1000^3 .

Strong scaling at 1000^3 . Efficiency $E_n=S_n/(nS_1)$ (Table 2, Figure 1b): H100 holds 91.6% at $n_p=8$; A100 holds 94.2%. The one anomalous point is H100 $n_p=8$ at 256^3 in Figure 1a: each rank owns only a very small subdomain, so launch and MPI overhead become comparable to the useful compute payload. This is a kernel-too-small regime rather than evidence of a correctness problem, and the behaviour largely disappears by 512^3 . A100 $n_p=8$ large-size coverage is incomplete and data-consistency is not fully confirmed, so it is not drawn as a main curve in Figure 1a; the small-size points we did capture live in the same kernel-too-small regime. The dip at 2500^3 in the H100 $n_p=8$ curve is consistent with weaker cache reuse and a less favourable memory regime at that size; however, this point also required the special timing path described above and is interpreted cautiously.

Table 2: Strong scaling at 1000^3 . Extra overhead = $1 - E_n$.

n_p	A100 SU/s	E_n	H100 SU/s	E_n	H100 extra overhead
1	3.514×10^{10}	100.0%	6.669×10^{10}	100.0%	0.0%
2	6.963×10^{10}	99.1%	1.316×10^{11}	98.6%	1.4%
4	1.349×10^{11}	96.0%	2.551×10^{11}	95.6%	4.4%
8	2.648×10^{11}	94.2%	4.885×10^{11}	91.6%	8.4%

Weak scaling (discussion) and communication. A dedicated weak-scaling sweep was not feasible. The 1000^3 strong-scaling series does not replace weak scaling, but it provides a useful communication anchor: as the per-rank subdomain shrinks, the additional scaling loss remains modest (8.4% on H100 and 5.8% on A100 at $n_p=8$). NSYS traces (Appendix B) report `MPI_Waitall` average call durations of 1.3–2.9 ms on H100 $n_p=4/8$. With two `MPI_Waitall` calls per step (one on the receive set, one on the send set), per-step wait totals ≈ 2.6 –5.8 ms while interior time is only 3.5/1.75 ms; on H100 $n_p=4$ the wait-fraction metric reads 40.4% yet $E_4=95.6\%$. This

gap, which plausibly reflects the dual-stream pipeline hiding most waits behind interior compute, collapses at $n_p=8$ where per-step wait exceeds interior time and E_8 falls to 91.6%.

Decomposition and surface-to-volume. At every GPU count we used the most symmetric decomposition available to minimise the surface-to-volume ratio and hence communication volume per step. At $n_p=8$ we also measured two asymmetric alternatives: $1 \times 2 \times 4$ lost $\approx 5\%$ and $1 \times 1 \times 8$ lost $\approx 14\%$ at 1000^3 relative to $2 \times 2 \times 2$, matching the predicted face-area ratios ($1 \times 1 \times 8$ has $\approx 2.3 \times$ the face area of the cube). This validates the symmetric-decomposition rule in our measurement conventions.

3 Further work: scaling to many nodes

Intra-node vs inter-node. Intra-node halos cross NVLink at 600–900 GB/s with $\sim \mu\text{s}$ latency. Across nodes, traffic follows PCIe $\rightarrow$ NIC $\rightarrow$ IB $\rightarrow$ NIC $\rightarrow$ PCIe at typically 25 GB/s per NDR-200 link with $\mathcal{O}(1-2 \mu\text{s})$ latency plus software stack overhead. At the measured anchor (total problem 1000^3 , $n_p=8$, $2 \times 2 \times 2$ decomposition) each rank owns a 500^3 subdomain and one face is $500^2 \times 8 \text{ B} \approx 2 \text{ MB}$: transferring one face takes $\sim 3 \mu\text{s}$ over NVLink, $\sim 80 \mu\text{s}$ over IB. Against a measured H100 $n_p=8$ interior of 1.75 ms, a $30 \times$ slower fabric still fits under the overlap ceiling, but the margin collapses.

A simple per-step model captures this: $T_{\text{step}} \approx \max(T_{\text{int}}, T_{\text{halo}}) + \epsilon_{\text{latency}}$, where $T_{\text{halo}} \approx V_{\text{face}}/B_{\text{fabric}} + L_{\text{fabric}}$ and V_{face} is the per-face halo volume. At this anchor, $T_{\text{int}} \gg T_{\text{halo}}$ intra-node, so the pipeline hides communication; as the effective fabric bandwidth drops, the crossover $T_{\text{halo}}=T_{\text{int}}$ determines the onset of visible efficiency loss. (This is an optimistic single-face model and ignores multi-face contention and imperfect overlap.)

Indicative efficiency forecasts (order-of-magnitude). Using Appendix B only as a coarse anchor, we infer that the exposed communication fraction is small on H100 $n_p=2$ and grows materially by $n_p=8$ as exposed communication time becomes comparable T_{int} . Extrapolating the same $T_{\text{halo}}/T_{\text{int}}$ ratio to a $2 \times 2 \times 2$ -per-node layout of H100s at $1000^3/\text{GPU}$:

- 2–4 nodes: about 80–85% efficiency. Only one Cartesian axis becomes inter-node in the smallest multi-node layouts.
- 16–64 nodes: roughly 60–70%. IB saturates at $\sim 20 \text{ GB/s}$ per link, so $T_{\text{halo}} \sim T_{\text{int}}$.
- 100+ nodes ($> 800 \text{ GPUs}$): at most $\sim 40\%$ unless per-rank problem grows; latency dominates and any global reductions pile on.

These percentages are indicative rather than model-derived predictions; they report the $T_{\text{halo}}/T_{\text{int}}$ ratios implied by the bandwidth / latency figures above as an order-of-magnitude guide only.

Code changes to reach that scale. (i) **Persistent MPI requests** (`MPI_Send_init/Start`) remove request-object construction from the hot loop. (ii) **CUDA Graphs** capture the 8-phase body once and replay; the $\approx 10 \mu\text{s}$ launch saving compounds on small per-rank problems. This is the optimisation we rejected for single-node and would adopt here. (iii) **Temporal blocking**: sweep K steps before exchanging halos deepened by K . B_{eff} drops to $\sim 48/K$ and halo volume grows by $K \times$ per exchange but is amortised over K steps, so net communication pressure drops $\sim K \times$. For $K=4$ this could materially improve many-node scalability, at the cost of a K -deep ghost region and careful handling of the time-dependent boundary damping. (iv) **Topology-aware placement** via `MPI_Dist_graph_create_adjacent` on the 3-D Cartesian neighbour graph so the neighbouring Cartesian ranks are mapped to nearby network locations. (v) **NCCL or tree-reduction** for any future global reductions.

Beyond the solver. At $\mathcal{O}(1000)$ GPUs the CPU reference path no longer fits in one node of host RAM, so correctness would move to computed invariants (energy, analytic Gaussian-pulse solution). Checkpoint frequency must become adaptive. IO is out of scope by brief; in production it would be the next limit.

A Optimisation A/B table

Each row aggregates 3–5 repeat runs. Δ is the mean percentage change of best-steady-state SU/s against the control.

Switch	Rationale	Measured at	A100 Δ	H100 Δ	Decision
Boundary-split kernels	remove interior early-return	$n_p=2$	+4.51%	+22.18%	adopted
Block shape (32, 4, 4)	warp-coalesced k + L2 x -reuse	$n_p=1$	+6.03%	+0.04%	adopted (default)
Shared-memory tiling	reduce global reads	$n_p=1$	−2.32%	−4.59%	rejected
MPI mode host vs device	CUDA-aware vs staging	$n_p=2$	host −10.13%	host −3.23%	adopted device
OpenMPI ob1 stack	alternative BTL	$n_p=2$	−0.08%	−41.45%	rejected
MPI.Waitsome loop	earlier unpack	$n_p=2$	−0.03%	−0.56%	rejected
Damping branchless	avoid warp divergence	$n_p=1$	−0.09%	≈ 0	rejected
z -padding (128 B align)	coalesce alignment	$n_p=1$	−0.37%	−0.51%	rejected

B Nsight Systems communication table

Totals over 20 profiled steps at 1000³. Wait-fraction = $T_{\text{wait,tot}}/(T_{\text{int,tot}}+T_{\text{wait,tot}})$; overlap proxy = $\max(0, 1 - T_{\text{wait,tot}}/T_{\text{int,tot}})$.

GPU	n_p	interior (ms)	Waitall avg (ms)	Waitall tot (ms)	wait frac.	overlap proxy
A100	2	13.56	0.24	9.68	3.2%	96.4%
A100	4	6.77	0.56	45.07	13.3%	83.4%
H100	2	7.17	0.47	18.81	10.9%	86.9%
H100	4	3.51	1.28	102.55	40.4%	27.0%
H100	8	1.75	2.89	462.28	75.3%	0.0%

Wait-fraction is a wall-clock waiting-behaviour metric rather than a true communication-cost share: on H100 $n_p=4$ it reads 40.4% while strong-scaling loss is only 4.4%, because much of the wait occurs while the interior kernel is still executing on `compute_stream`. (Note: `Waitall tot` values at larger n_p reflect Nsight Systems aggregating metrics across multiple profiled ranks, scaling the total call count beyond a single rank’s 40 calls; the derived `wait frac` and `avg` columns remain mathematically representative. Interpret `avg` as the mean duration of a single `MPI.Waitall` call, not a per-step wait; each step issues two such calls.)

C Note on the 2500³ reference measurement

The unmodified driver stores two mirrored host simulations (CPU reference + CUDA state), across six `double` arrays (`u_prev, u_now, u_next, cs2, damp, sos`) ≈ 96 B per lattice point. At 2500³ spread over 8 ranks on one node, this totals roughly 1.5 TB of host memory, exceeding the 512 GB per-node RAM.

For the 2500³ reference point only, we made two targeted adjustments (not part of the submitted source):

- `main.cpp`: under `-skip_cpu 1`, skip the CPU-mirror allocation and clone path.
- `wave_cuda.cu`: a small initialiser that reconstructs the initial condition directly on device from the same analytic parameters, removing the dependence on a host copy. This runs once before the time loop and does not affect per-step timing.

GPU kernels and MPI path are unchanged from the submission. The measurement reflects device-path performance; we report it as a reference only, and clearly label it in Figure 1a and the surrounding text.

References

- [1] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. White paper, 2020. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [2] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*. White paper, 2022. <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>
- [3] S. Williams, A. Waterman, and D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009. <https://doi.org/10.1145/1498765.1498785>
- [4] Open MPI Project. *Open MPI v4.1.x documentation*. <https://docs.open-mpi.org/en/v4.1.x/>
- [5] NVIDIA Corporation. *Nsight Systems User Guide*. <https://docs.nvidia.com/nsight-systems/UserGuide/>